

IoT Lab Assignment 6

RSA Protected Socket Programming

Part 1 – Introduction

When you use SSH to connect your Raspberry Pi via Ethernet, your computer creates a secure tunnel that transmits commands and receives responses. In this lab, you will build a secure tunnel using Python. First, you will set up a loopback socket connection between two programs on your machine. Although this connection is not secure, it demonstrates how two programs can communicate with each other. Next, you will implement a small-scale 32-bit RSA encryption algorithm from scratch. Then, by integrating RSA with socket programming, you will develop a custom SSL protocol. Finally, you will perform a man-in-the-middle (MITM) attack on your custom SSL to experience firsthand the challenges of securing data transmission. A zip file containing the incomplete Python scripts for all four parts of the lab is provided and can be downloaded from the resources section on pizza.

Part 2 – Socket Programming

Unzip the "Lab6 - Secure Socket.zip" file and navigate to the "Lab6 - Secure Socket/Lab6.1 Socket/Problem" directory. In this folder, you'll find the working scripts for Client.py and Server.py. These scripts include auxiliary code for handling input arguments and exceptions while concentrating on socket communications. Inline comments will help you understand the code structure.

To test the scripts, open two terminal windows. In one terminal, launch the server script first, and then run the client script in the other terminal.

To start the server:

```
python3 Server.py -p <a port number>
```

To start the client:

```
python3 Client.py -p <a port number> [-s server_ip_or_hostname]
```

If the server and client are on the same machine, you can use the localhost IP address (127.0.0.1) for the server. If the server is running on a different device, such as your Raspberry Pi, provide that device's IP address or hostname.

Part 3 – RSA

Navigate to the "Lab6 - Secure Socket/Lab6.2 RSA/Problem" folder. Your objective is to complete the RSA-offline.py script by implementing the RSA key generation, encryption, and decryption processes, following the guidance provided in the comments.

Additionally, a helpful script named Number_Package.py is included. This script contains all the RSA-related functions you might need. While you do not need to understand the inner workings of each function, you should be aware of their availability.

```
## all the below functions are optimized for both PC and RPi
# prime number related
is_prime(a)          # check whether "a" is a prime number
find_prime_smaller_than_k(k)
find_prime_greater_than_k(k)
# operations in GF
mult_inv_mod_N(a, N) # a^(-1) mod N
exp_mod(a, b, N)     # a^b mod N
```

Please do not use any other third-party libraries or packages to complete this task.

Here are some realizations of RSA. You can use them to verify the correctness of your code.

Example 1	p	1637842597
	q	2657124709
	N	4351952033941429273
	e	65537
	d	327573422070677585
	m	2146210639644006206
	c	768594581363479181
Example 2	p	1900493143
	q	2394474161
	N	4550681724071178023
	e	65537
	d	4530892222390974593
	m	914533224344639919
	c	3295824207614022858
Example 3	p	1907263091
	q	2387704201
	N	4553980094792945291

	e	65537
	d	1438314751868679473
	m	3135653864232958471
	c	3360043531845750575

Fill the missing numbers in the following Table to test your understanding of RSA.

Exercise 1 Undergrad and Graduate Students	p	1998847601
	q	2296119677
	N	
	e	65537
	d	
	m	10001
	c	
Exercise 2 Undergrad and Graduate Students	N	4400141856896539081
	d	2764011776369096961
	m	
	c	2503673653585535070
Exercise 3 Graduate Students Only	p	1940829017
	q	
	N	4568979870268667641
	e	
	d	610100779569588057

Part 4 – RSA Protected Socket

In this section, you will integrate the work from Parts 2 and 3 to implement RSA-protected socket communication. Navigate to the "Lab6 - Secure Socket/Lab6.3 RSA Protected Socket/Problem" folder and complete both the Client.py and Server.py scripts by following the inline comments.

Note: If you run the program on a Raspberry Pi, please be aware that due to its limited computational power, responses may take up to 6 seconds.

```
Trying to connect to 127.0.0.1:1234
Server's key: 818942888754917992
server msg: Hello, this is the server.
```

Figure 1 Client's Example Output

```

Server is listening to port 1234
=====
waiting for connection
Got connection from ('127.0.0.1', 64541)
server chooses key: 818942888754917992
Client msg: This is the client. Let's follow your selected key to communicate!
=====

```

Figure 2 Server's Example Output

We provide a simple function called **bytearray_xor(key, msg_bytes)** that performs both encryption and decryption. This function operates by applying a bitwise exclusive OR (XOR) between the provided key and the secret message bytes. Thanks to the symmetric nature of XOR (i.e., applying the key twice returns the original message), the same function can be used for both processes.

P.S. Modern encryption algorithms typically avoid reusing keys to enhance security. However, to keep things simple and avoid delving into more complex key generation methods (such as those in DES, AES, or RC4), we have chosen this approach. If you're interested, you might explore how a known-plaintext attack could potentially expose the key.

Part 5 – MITM Attack (Graduate Students)

In this task, you will perform a Man-In-The-Middle (MITM) attack on the RSA-protected socket connection developed in the previous section. Navigate to the "Lab6 - Secure Socket/Lab6.4 MITM Attack/Problem" directory to find the incomplete Mallory.py script. The attack logic follows the strategy illustrated in Figure 3, and your goal is to complete the script based on that guidance.

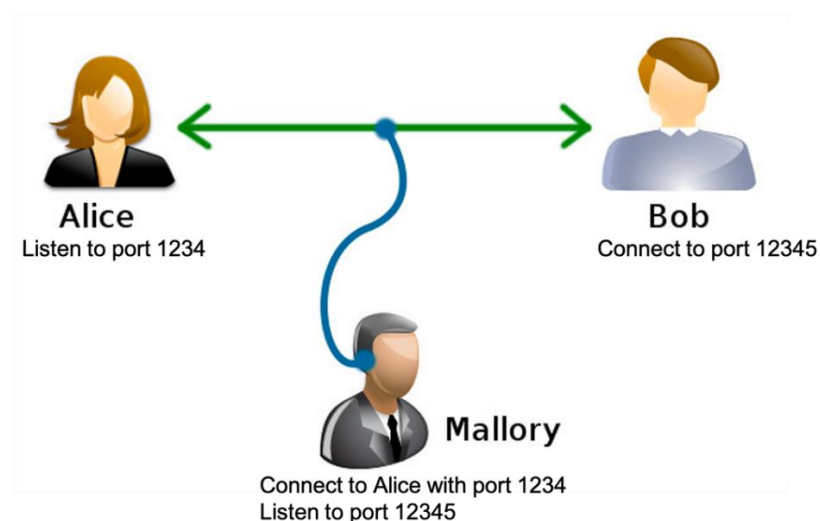


Figure 3 MITM Attack Diagram

MITM Attack on RSA-Protected Socket Communication

In this scenario, Alice, the legitimate server, listens on port 1234. Bob, the unsuspecting client, has been misled by Mallory into believing that the server is running on port 12345. Mallory, the attacker, simultaneously establishes two connections—one with Alice and one with Bob—with the goal of intercepting the communication between them without their knowledge.

Attack Process:

1. **Key Negotiation:**
Mallory negotiates an RSA key with Alice while aligning with Bob's RSA key selection.
2. **Key Interception:**
When Alice sends her selected encryption key to Mallory, he stores a copy and forwards the identical key to Bob.
3. **Message Relay:**
As messages are exchanged between Alice and Bob using the shared key, Mallory relays them unchanged. Since he has a copy of the key, he can decrypt and read all the transmitted data.

Execution Order:

Start the server (Alice) first, then run Mallory's script, and finally launch the client (Bob).

Figure 4 depicts a successful attack scenario.

```

yuwang@uone-mbp-16:/Volumes/GoogleDrive/My Drive/RPI/TA/21Spring/ECSE4660/Labs/L
ab6 - Secure Socket/forTA/Lab6 - Secure Socket/Lab6.3 RSA Protected Socket/Solut
ion$python3 Server-sol.py -p 1234
Server is listening to port 1234

=====
waiting for connection
Got connection from ('127.0.0.1', 52852)
server chooses key: 3847625963552399758
Client msg: This is the client. Let's follow your selected key to communicate!
=====

yuwang@uone-mbp-16:/Volumes/GoogleDrive/My Drive/RPI/TA/21Spring/ECSE4660/Labs/L
ab6 - Secure Socket/forTA/Lab6 - Secure Socket/Lab6.3 RSA Protected Socket/Solut
ion$python3 Client-sol.py -p 12345
Trying to connect to 127.0.0.1:12345
Server's key: 3847625963552399758
server msg: Hello, this is the server.
=====

yuwang@uone-mbp-16:/Volumes/GoogleDrive/My Drive/RPI/TA/21Spring/ECSE4660/Labs/L
ab6 - Secure Socket/forTA/Lab6 - Secure Socket/Lab6.4 MITM Attack/Solution$pytho
n3 Mallory-sol.py -p 1234 -c 12345
Mallory is listening to port 12345

=====
waiting for victim to connect
Got connection from victim ('127.0.0.1', 52851)
Server's key: 3847625963552399758
Victim -> Server: This is the client. Let's follow your selected key to communic
ate!
Server -> Client: Hello, this is the server.
=====

```

Server

Victim

Mallory

Figure 4 Example Output of a Success MITM Attack

Part 6 – Deliverables

For Both Undergraduate and Graduate Students:

1. Submit complete Python scripts for Parts 3 and 4.
2. Include terminal screenshots for Parts 2, 3, and 4 in your report to demonstrate task completion.
3. Complete the provided table in Part 3 and attach it to your report.
4. Answer the following question in your report:
Is it advisable to use RSA for transmitting large amounts of data (for example, encrypting a 1GB file)? Explain your reasoning.

For Graduate Students Only:

5. Complete the Mallory script for Part 5.
6. Provide terminal screenshots that verify a successful MITM attack (similar to Figure 4).
7. Answer the following question in your report:
How can the MITM attack be defended against? Provide a brief explanation.

